

L2: Querying Data

RECAP from Lab #1

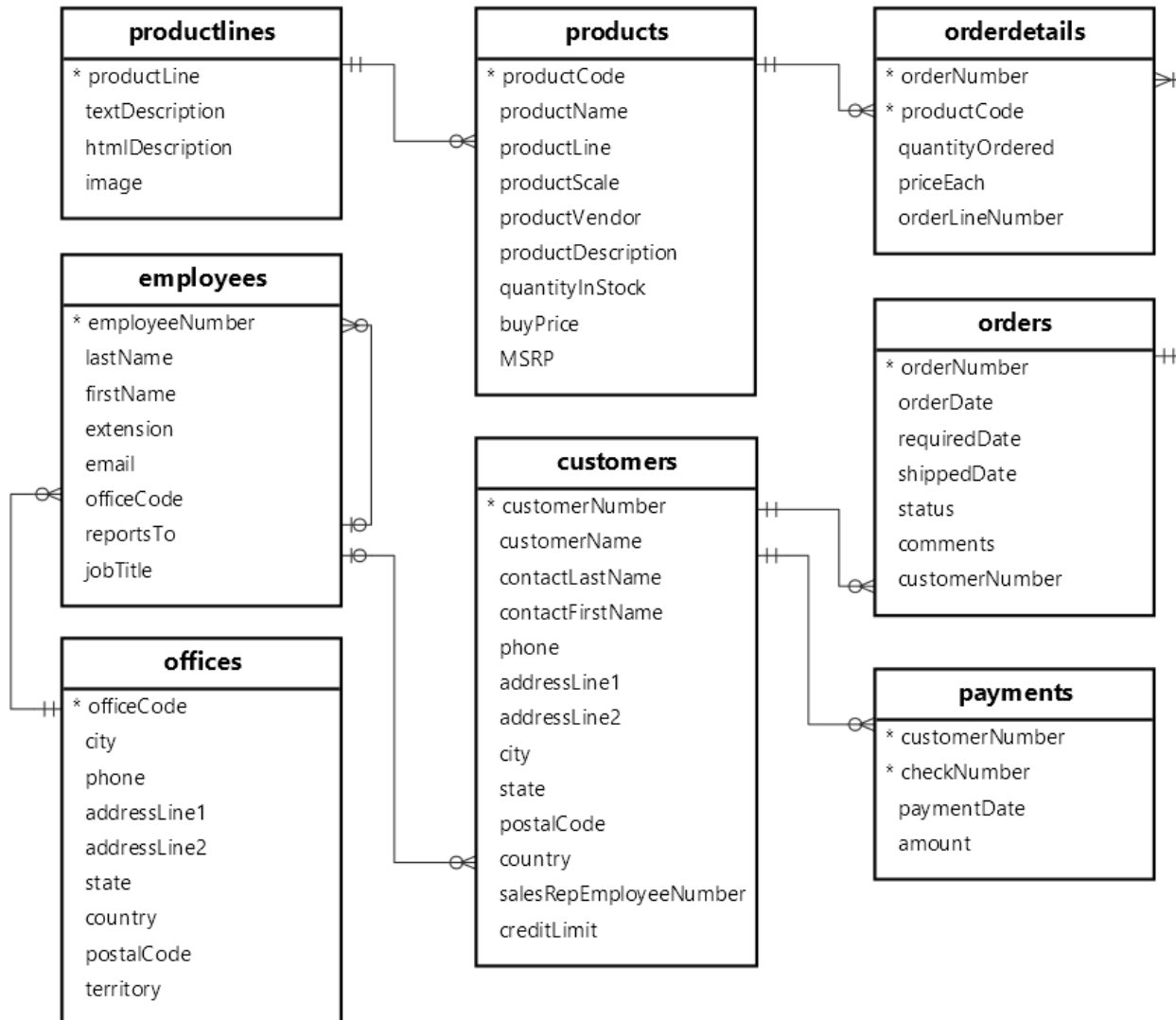
- Add new column, Alter column and Delete column
- Define Primary Key (PK) and Foreign Key (FK)
- Create and Delete constraints

```
CREATE TABLE Trainer (  
    trainerID INT NOT NULL,  
    trainer_name VARCHAR(50),  
    hometown VARCHAR(50),  
    registered_date DATE,  
    PRIMARY KEY (trainerID)  
);
```

```
ALTER TABLE vehicles  
ADD model VARCHAR(100) NOT NULL  
AFTER year;
```

```
ALTER TABLE suppliers  
ADD CONSTRAINT uc_name_address  
UNIQUE (name,address);
```

2.1 Example Database



- **customers**: stores customer's data.
- **products**: stores a list of scale model cars.
- **productlines**: stores a list of product lines.
- **orders**: stores sales orders placed by customers.
- **orderdetails**: stores sales order line items for every sales order.
- **payments**: stores payments made by customers based on their accounts.
- **employees**: stores employee information and the organization structure such as who reports to whom.
- **offices**: stores sales office data.

Load Database

By CMD

```
source path_to_database.sql;
```

By Workbench

Run the SQL script

```
show databases;
```

Database

classicmodels

information_schema

mysql

performance_schema

sys

test

test2

test3

test4

test5

test6

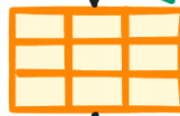
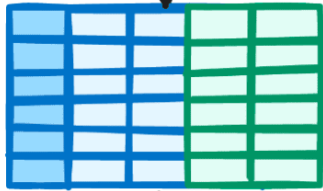
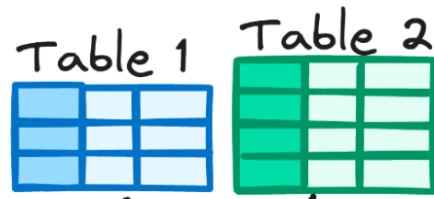
2.2 SELECT Statement

Syntax

```
SELECT  column_list  
FROM    Source  
[WHERE  condition]  
[GROUP BY grouping_expression]  
[HAVING grouping_condition]  
[ORDER  BY expression ASC|DESC]
```

- Retrieve data from a database
- column_list = one or more columns separated by commas
- * (asterisk) = list all columns





SELECT DISTINCT

Table1.*,
Table2.*

STEP 5

FROM Table1
JOIN Table2

ON Table1.col1 = Table2.col2

STEP 1

WHERE constraint_expression

STEP 2

GROUP BY [columns]

STEP 3

HAVING constraint_expression

STEP 4

ORDER BY [columns]

STEP 6

LIMIT count

STEP 7

SELECT Statement

Example

```
SELECT * FROM payments;
```

customerNumber	checkNumber	paymentDate	amount
103	HQ336336	2004-10-19	6066.78
103	JM555205	2003-06-05	14571.44
103	OM314933	2004-12-18	1676.14
112	BO864823	2004-12-17	14191.12
112	HQ55022	2003-06-06	32641.98
112	ND748579	2004-08-20	33347.88
114	GG31455	2003-05-20	45864.03
114	MA765515	2004-12-15	82261.22
114	NP603840	2003-05-31	7565.08
114	NR27552	2004-03-10	44894.74
119	DB933704	2004-11-14	19501.82
119	LN373447	2004-08-08	47924.19
119	NG94694	2005-02-22	49523.67

Example

```
SELECT customerNumber, amount  
FROM payments;
```

customerNumber	amount
103	6066.78
103	14571.44
103	1676.14
112	14191.12
112	32641.98
112	33347.88
114	45864.03
114	82261.22
114	7565.08

2.2 Column alias

Syntax

```
SELECT expression AS column_alias;
```

```
SELECT column_list  
FROM Source  
[WHERE condition]  
[GROUP BY expression]  
[HAVING condition]  
[ORDER BY expression ASC|DESC]
```

- MySQL uses the expression specified in the SELECT clause as the column name of the result set.
- Concatenate operation can be used for the char domain.
- Arithmetic operators (e.g., +, -, *, /) and parentheses can be used for integer domain.

Column alias

Example

```
SELECT
    CONCAT(firstName, ' ', lastName) AS 'Full name'
FROM
    employees;
```

Full name
Diane Murphy
Mary Patterson
Jeff Firrelli
William Patterson
Gerard Bondur

Example

```
SELECT
    priceEach * quantityOrdered AS 'Total Price'
FROM
    orderdetails;
```

Total Price
4080.00
2754.50
1660.12
1729.21
2701.50

2.3 Search Condition

Example

```
SELECT lastname, firstname, jobtitle
FROM employees
WHERE jobtitle = 'Sales Rep';
```

```
SELECT column_list
FROM Source
[WHERE condition]
[GROUP BY expression]
[HAVING condition]
[ORDER BY expression ASC|DESC]
```

lastname	firstname	jobtitle
Jennings	Leslie	Sales Rep
Thompson	Leslie	Sales Rep
Firrelli	Julie	Sales Rep
Patterson	Steve	Sales Rep
Tseng	Foon Yue	Sales Rep
Vanauf	George	Sales Rep
Bondur	Loui	Sales Rep
Hernandez	Gerard	Sales Rep

Comparison Operators

Operator	Description
=	Equal to. You can use it with almost any data type.
<> or !=	Not equal to
<	Less than. You typically use it with numeric and date/time data types.
>	Greater than.
<=	Less than or equal to
>=	Greater than or equal to

Conditions Merging Operators

Operator	Description
AND / OR	The AND/OR operator is a logical operator returns 1, 0, or NULL
IN NOT IN	The IN operator is functionally equivalent to a combination of multiple OR operators . <code>value IN (value1, value2, value3, ...)</code>
BETWEEN	The BETWEEN operator is a logical operator that specifies whether a value is in a range or not . <code>value BETWEEN low AND high;</code>
LIKE	The LIKE operator is a logical operator that tests whether a string contains a specified pattern or not . <code>expression LIKE pattern ESCAPE escape_character</code> • The percentage (%) wildcard matches any string of zero or more characters. • The underscore (_) wildcard matches any single character.

Search Condition

Example

```
SELECT
    customerName,
    country
FROM
    customers
WHERE
    country IN ('USA' , 'France');
```

customerName	country
Atelier graphique	France
Signal Gift Stores	USA
La Rochelle Gifts	France
Mini Gifts Distributors Ltd.	USA
Mini Wheels Co.	USA
Land of Toys Inc.	USA

Example

```
SELECT
    customerNumber,
    customerName
FROM
    customers
WHERE
    customerNumber
    BETWEEN 120 AND 125;
```

customerNumber	customerName
121	Baane Mini Imports
124	Mini Gifts Distributors Ltd.
125	Havel & Zbyszek Co

Search Condition

Example

```
SELECT
    lastName,
    firstName
FROM
    employees
WHERE
    firstname LIKE 'T_m';
```

```
WHERE
    firstName LIKE 'a%';
```

```
WHERE
    lastname LIKE '%on%';
```



lastName	firstName
King	Tom



lastName	firstName
Bow	Anthony
Fixter	Andy



lastName	firstName
Patterson	Mary
Patterson	William
Bondur	Gerard
Thompson	Leslie
Patterson	Steve

Search Condition

Sometimes the pattern may contain the wildcard characters
e.g., 10%, _20, etc.

Example: Backslash (\)

```
WHERE  
    productCode LIKE '%\_20%';
```

Example: ESCAPE clause

```
WHERE  
    productCode LIKE '%$_20%'  
    ESCAPE '$';
```

productCode	productName
S10_2016	1996 Moto Guzzi 1100i
S24_2000	1960 BSA Gold Star DBD34
S24_2011	18th century schooner
S24_2022	1938 Cadillac V-16 Preside...
S700_2047	HMS Bounty

2.4 Group By

Example

```
SELECT status
FROM orders
GROUP BY status;
```

```
SELECT column_list
FROM Source
[WHERE condition]
[GROUP BY expression]
[HAVING condition]
[ORDER BY expression ASC|DESC]
```

orderNumber	orderDate	requiredDate	shippedDate	status	col
10163	2003-10-20	2003-10-27	2003-10-24	Shipped	HULL
10164	2003-10-21	2003-10-30	2003-10-23	Resolved	Thi
10165	2003-10-22	2003-10-31	2003-12-26	Shipped	Thi
10166	2003-10-21	2003-10-30	2003-10-27	Shipped	HULL
10167	2003-10-23	2003-10-30	HULL	Cancell...	Cus
10168	2003-10-28	2003-11-03	2003-11-01	Shipped	HULL
10169	2003-11-04	2003-11-14	2003-11-09	Shipped	HULL

status
Shipped
Resolved
Cancelled
On Hold
Disputed
In Process

Group By (with aggregate fcn.)

Example

```
SELECT
  status,
  COUNT(*)
FROM
  orders
GROUP BY
  status;
```

status	COUNT(*)
Shipped	303
Resolved	4
Cancelled	6
On Hold	4
Disputed	3
In Process	6

Aggregate Function

Aggregate functions are used to summarize data.

Functions	Description
COUNT	Count number of not-null values of an attribute
MAX	Find highest value in a table
MIN	Find lowest value in a table
SUM	Compute total sum for any specified attribute
AVG	Compute the average value of any specified attribute
STD	Compute the population standard deviation

See more: <https://dev.mysql.com/doc/refman/8.0/en/aggregate-functions.html>

2.5 Having

```
SELECT column_list
FROM Source
[WHERE condition]
[GROUP BY expression]
[HAVING condition]
[ORDER BY expression ASC|DESC]
```

Example

```
SELECT    ordernumber,
          SUM(quantityOrdered) AS itemCount,
          SUM(priceeach * quantityOrdered) AS total
FROM      orderdetails
GROUP BY  ordernumber
HAVING    total > 1000;
```

ordernumber	itemCount	total
10100	151	10223.83
10101	142	10549.01
10102	80	5494.78
10103	541	50218.95
10104	443	40206.20

Where vs Having

```
SELECT column_list
FROM Source
[WHERE condition]
[GROUP BY expression]
[HAVING condition]
[ORDER BY expression ASC|DESC]
```

Criteria	WHERE Clause	HAVING Clause
Purpose	Filters rows before aggregation.	Filters rows after aggregation.
Used With	Works with individual rows.	Works with grouped or aggregated data.
Applicable To	Columns in the table (individual records).	Aggregated results (e.g., SUM(), COUNT()).
Stage of Query Execution	Filters data before GROUP BY.	Filters data after GROUP BY.
Aggregate Functions	Cannot be used with aggregate functions.	Can be used with aggregate functions.
Efficiency	Generally more efficient, as it filters data earlier.	Can be less efficient, as it filters after aggregation.
Example Usage	WHERE column_name = value	HAVING aggregate_function(column_name) > value
Order in Query	Appears before GROUP BY.	Appears after GROUP BY.

2.6 Order By

Example

```
SELECT
    contactLastname,
    contactFirstname
FROM
    customers
ORDER BY
    contactLastname DESC,
    contactFirstname ASC;
```

```
SELECT column_list
FROM Source
[WHERE condition]
[GROUP BY expression]
[HAVING condition]
[ORDER BY expression ASC|DESC]
```

contactLastname	contactFirstname
Young	Dorothy
Young	Jeff
Young	Julie
Young	Mary
Yoshido	Juri
Walker	Brydey
Victorino	Wendy
Urs	Braun
Tseng	Jerry
Tonini	Daniel
Thompson	Steve
Thompson	Valarie
Taylor	Leslie

2.7 Aggregation Function

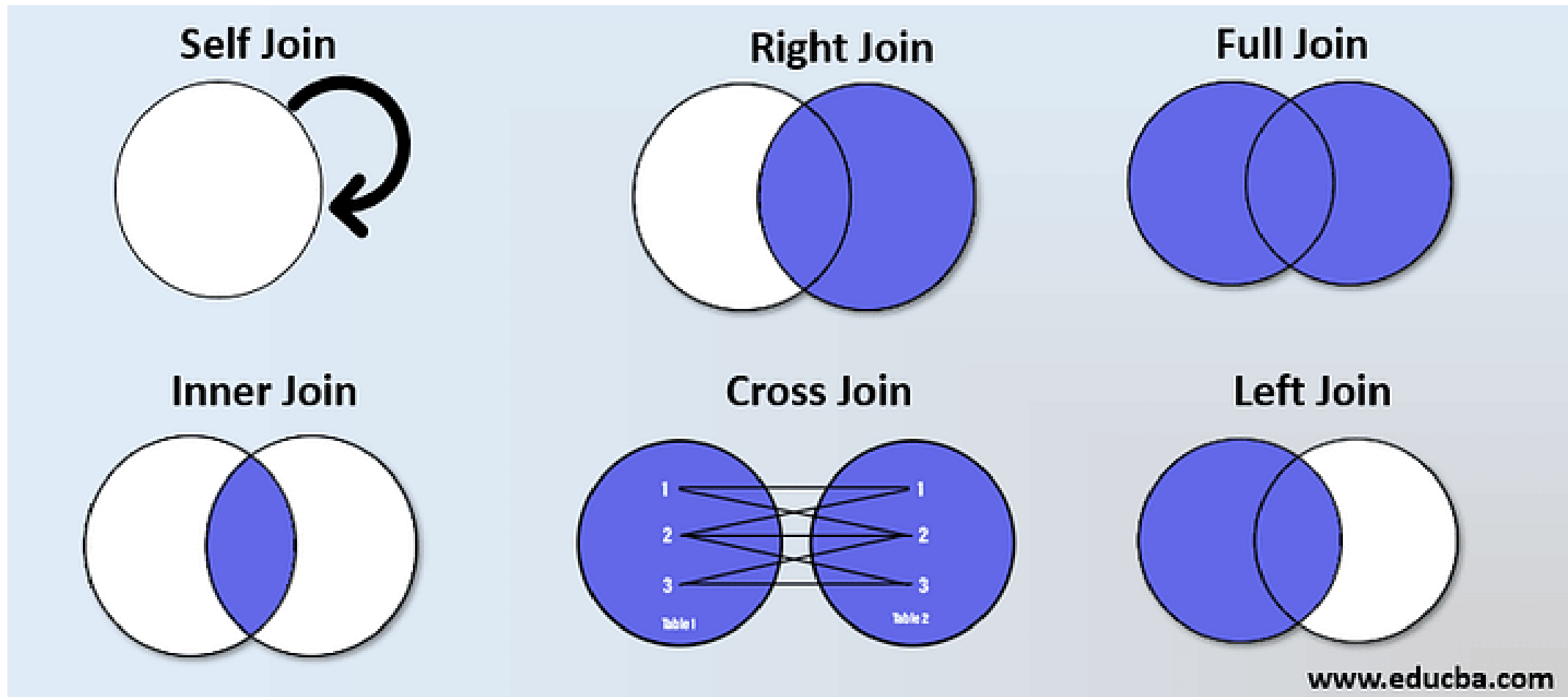
Example

```
SELECT
    productLine,
    AVG(buyPrice)
FROM
    products
GROUP BY productLine
ORDER BY productLine;
```

productLine	AVG(buyPrice)
Classic Cars	64.446316
Motorcycles	50.685385
Planes	49.629167
Ships	47.007778
Trains	43.923333
Trucks and Buses	56.329091

See more function: <https://www.mysqltutorial.org/mysql-aggregate-functions/>

2.8 Join Operations



Join Operations

Example1: Inner Join

```
SELECT *  
FROM members INNER JOIN committees  
USING (name) ;
```

Table: members

member_id	name
1	John
2	Jane
3	Mary
4	David
5	Amelia

Join

Table: committees

committee_id	name
1	John
2	Mary
3	Amelia
4	Joe



name	member_id	committee_id
John	1	1
Mary	3	2
Amelia	5	3

Join Operations

Example2: Inner Join

```
SELECT *  
FROM members INNER JOIN committees  
ON members.name = committees.name;
```

Table: members

member_id	name
1	John
2	Jane
3	Mary
4	David
5	Amelia

Join

Table: committees

committee_id	name
1	John
2	Mary
3	Amelia
4	Joe



member_id	name	committee_id	name
1	John	1	John
3	Mary	2	Mary
5	Amelia	3	Amelia

Join Operations

Example: Left

```
SELECT *  
FROM members LEFT JOIN committees  
USING (name) ;
```

name	member_id	committee_id
John	1	1
Jane	2	NULL
Mary	3	2
David	4	NULL
Amelia	5	3

Example: Right

```
SELECT *  
FROM members RIGHT JOIN committees  
USING (name) ;
```

name	committee_id	member_id
John	1	1
Mary	2	3
Amelia	3	5
Joe	4	NULL

Join Operations

Example: Cross

```
SELECT *  
FROM members CROSS JOIN committees;
```

member_id	name	committee_id	name
1	John	4	Joe
1	John	3	Amelia
1	John	2	Mary
1	John	1	John
2	Jane	4	Joe
2	Jane	3	Amelia
2	Jane	2	Mary
2	Jane	1	John
3	Mary	4	Joe
3	Mary	3	Amelia
3	Mary	2	Mary
3	Mary	1	John

Join Operations

Example: Full Outer Join

```
SELECT name FROM members  
UNION  
SELECT name FROM committees;
```

Table: members

member_id	name
1	John
2	Jane
3	Mary
4	David
5	Amelia

Table: committees

committee_id	name
1	John
2	Mary
3	Amelia
4	Joe

Join



name
John
Jane
Mary
David
Amelia
Joe

Note. MySQL hasn't supported the FULL OUTER JOIN syntax yet.

Join Operations

Example: [Self Join](#)

```
SELECT
    CONCAT(m.lastName, ', ', m.firstName) AS Manager,
    CONCAT(e.lastName, ', ', e.firstName) AS 'Direct report'
FROM
    employees e INNER JOIN employees m
ON
    m.employeeNumber = e.reportsTo
ORDER BY
    Manager;
```

employeeNumber	lastName	firstName	reportsTo
1002	Murphy	Diane	HULL
1056	Patterson	Mary	1002
1076	Firrelli	Jeff	1002
1088	Patterson	William	1056
1102	Bondur	Gerard	1056
1143	Bow	Anthony	1056



Manager	Direct report
Bondur, Gerard	Bondur, Loui
Bondur, Gerard	Hernandez, Gerard
Bondur, Gerard	Castillo, Pamela
Bondur, Gerard	Bott, Larry
Bondur, Gerard	Jones, Barry
Bondur, Gerard	Gerard, Martin
Bow, Anthony	Jennings, Leslie